AI_Trip_Planner-10.pdf

Shri Vile Parle Kelavani Mandal

Document Details

Submission ID
trn:oid:::9832:135211251

Submission Date
Apr 14, 2026, 11:08 PM GMT+5:30

Download Date
May 7, 2026, 10:32 PM GMT+5:30

File Name
AI_Trip_Planner-10.pdf

File Size
376.4 KB

6 Pages

3,521 Words

19,337 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

13 Not Cited or Quoted 4%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

6 Missing Citation 2%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
4% Publications
5% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **13** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **6** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
4% Publications
5% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Capella University on 2025-02-24 <1%
2	Submitted works	Virginia Commonwealth University on 2025-03-21 <1%
3	Internet	core.ac.uk <1%
4	Submitted works	University of Leeds on 2019-05-09 <1%
5	Publication	W.H. Schuler, C.J.A. Bastos-Filho, A.L.I. Oliveira. "A novel hybrid training method f... <1%
6	Internet	nzdr.ru <1%
7	Submitted works	IIT Delhi on 2015-12-17 <1%
8	Internet	re.public.polimi.it <1%
9	Submitted works	Virginia Commonwealth University on 2025-03-20 <1%
10	Internet	dspace.bracu.ac.bd <1%

11	Internet	
epochjournals.com		<1%
12	Internet	
www.geeksforgeeks.org		<1%
13	Internet	
www.iieta.org		<1%
14	Submitted works	
Amrita Vishwa Vidyapeetham on 2026-04-14		<1%
15	Internet	
9dok.org		<1%
16	Internet	
www.classace.io		<1%
17	Internet	
researchdatapod.com		<1%
18	Submitted works	
King's College on 2018-04-16		<1%

AI-Based Constraint-Aware Inter-City Trip Planner with Dynamic Re-Planning

Divyanshu Patil, Parth Dhadke, Vivek Sondagar, Ribsun Yadav
Department of Computer Science and Engineering
NMIMS Mumbai, India
Email: {divyanshupatil1109, dhadkeparthn, viveksondagar, ribsunyadav}@gmail.com

Mentor: Rama Varshney
Department of Computer Science and Engineering
NMIMS Mumbai, India
Email: rama.varshney@nmims.edu

***Abstract*—Travel between cities demands choices shaped by driving duration, fuel prices, tolls, traffic delays, and how tired a person gets behind the wheel. Old-style route apps usually chase the quickest line on a map - thanks to early algorithms like Dijkstra's - but skip daily hurdles drivers face. That means the computer's favorite road might look fine on screen yet fall apart on actual highways.**

A new approach to trip planning takes constraints into account by using weighted graphs to represent transport routes [4], combining them with methods for meeting multiple goals at once [3]. Instead of ignoring real-world limits, it works around fixed budgets, how long someone can drive, and required arrival times - key pieces in today's smart transit solutions [7].

A fresh path unfolds when routes adapt beyond simple shortcuts, weaving better choices into daily travel puzzles. Instead of chasing only the quickest line on a map, smarter turns emerge through balanced thinking. Real cities need systems that breathe with traffic pulses, not rigid formulas stuck in old logic. This way fits messy streets where delays shift by the minute. Judgment grows sharper when options reflect actual road rhythms. Life does not follow straight lines - neither should navigation.

***Index Terms*—Trip Planning, Constraint Satisfaction, Graph Algorithms, Optimization, Dynamic Routing**

I. Introduction

Getting people between cities well sits at the core of smart transport design [7], where picking best paths means juggling many practical limits. Older methods usually lean on basic route finders like Dijkstra's technique [1], along with A star [5], aiming only at one goal - be it miles covered or hours spent.

Still, actual trips come with extra hurdles like fuel prices, toll fees, tired drivers, or fixed timetables. Such details turn route planning into a challenge of balancing limits and competing goals [3]; sometimes the shortest path on paper becomes unrealistic once you hit the road.

Nowadays studies in transport look at tight-resource route choices [2], methods balancing multiple goals [3], along with ways to handle routing across vast networks [4]. With these tools, planners can better reflect actual limits while shaping decisions people face daily.

When things like traffic or personal choices shift mid-trip, finding new paths quickly matters. Because updates happen on the fly, systems rely on graph methods that adjust routes without delay [1].

A fresh look at trip planning takes shape here - blending route mapping with rule checking inside one working model. Built to handle actual limits like money and hours spent moving, it shapes paths that fit life's boundaries. Instead of treating restrictions as afterthoughts, they guide choices from the start. This approach connects navigation logic directly to what users can actually accept. Real conditions mold each outcome, not just ideal scenarios. Every output respects both map details and personal needs without ignoring either.

II. Literature Review

Out of all ways to map movement, following paths that twist like city streets shows what numbers can reveal when guided by older ideas. Instead of just theory, effort now tracks how flow really works, borrowing methods shaped in earlier trials.

- **Dijkstra's Algorithm [1]:** Starting from a single location, Dijkstra's method selects the neighboring path that is least expensive. Results are consistent if costs are positive. On that account GPS tools rely on those calculations frequently - but the process changes if external constraints are present.
- **A* Search Algorithm [5]:** As this logic includes predictions, it operates at a high velocity. Due to the approximations, the system is efficient. With large environments, paths are accurate.
- **Resource-Constrained Shortest Path [2]:** Here things shift once clocks tick down plus budgets shrink. Suddenly paths must bend when cash runs short or minutes fade away.
- **Multi-Objective Optimization [3]:** Slowing progress on price often follows attempts to rush arrival times - each gain leans on what it sacrifices. Pushed by competing targets, universal fixes crumble under pressure. Juggling separate objectives? Strength in one zone may bleed into loss elsewhere. Decisions tilt between cost and clock, neither guaranteed priority

- **Transportation Network Optimization [4]:** Out of nowhere, thinking differently about motion inside massive transit networks changes everything. Before you even leave, better preparation sharpens your route. While on the move, real-time tweaks keep things clear. Instead of guessing, live data mixes with forecasts so choices happen faster. As seconds count more, actions sync closely with current conditions. What used to rely only on static plans now leans heavily on unfolding events.
- **Multi-Constraint Routing [6]:** Back when folks first looked into multi-constraint routing, they came up with ways to handle paths under several limits at once - those ideas later became the base for today's smarter routing tools that pay attention to constraints. Though rough at first, those early approaches quietly shaped how such systems think about complex rules now.
- **Intelligent Transportation Systems [7]:** Real-time data shapes much of what ITS researchers explore, while predictions help spot patterns before they fully form. Efficiency gains come not just from numbers but through how routes adjust on their own mid-flow. User experience shifts when systems respond instead of stick to fixed plans.

III. System Architecture

The architecture (Fig. 1) is a pipeline of multiple modules.

A. Input Processing

As the process begins, the user provides data - if a person enters a destination, the system examines it. There are instances where the system receives a budget to review. After the system gathers those inputs, it organizes them. On that account the data is in a format that the next stages can use easily. With this structure, the data moves through the system. Due to the predictable forms, the process is efficient. In some cases the system fixes cost figures immediately. By doing this the system avoids mistakes. When a piece of information is correct, it moves to the next part. It is through this arrangement that the system functions.

B. Graph Construction

Locations form points on a map, linked by paths that connect one to another. Between every pair of spots, there exists a route marked with details like how far it is, how long it takes, also what it costs. A structure made up of these places and links acts as a stand-in for real movement networks. This setup draws from earlier work tracking how things move through connected systems. Each path carries values, shaping how trips are understood across the layout. [4], [8].

C. Preprocessing

Out in the field, numbers shape how links between points get their value -distance plays a part, fuel expenses chip in too, toll fees add on top. When mapping out possible paths, those values step forward to show which way might work best.

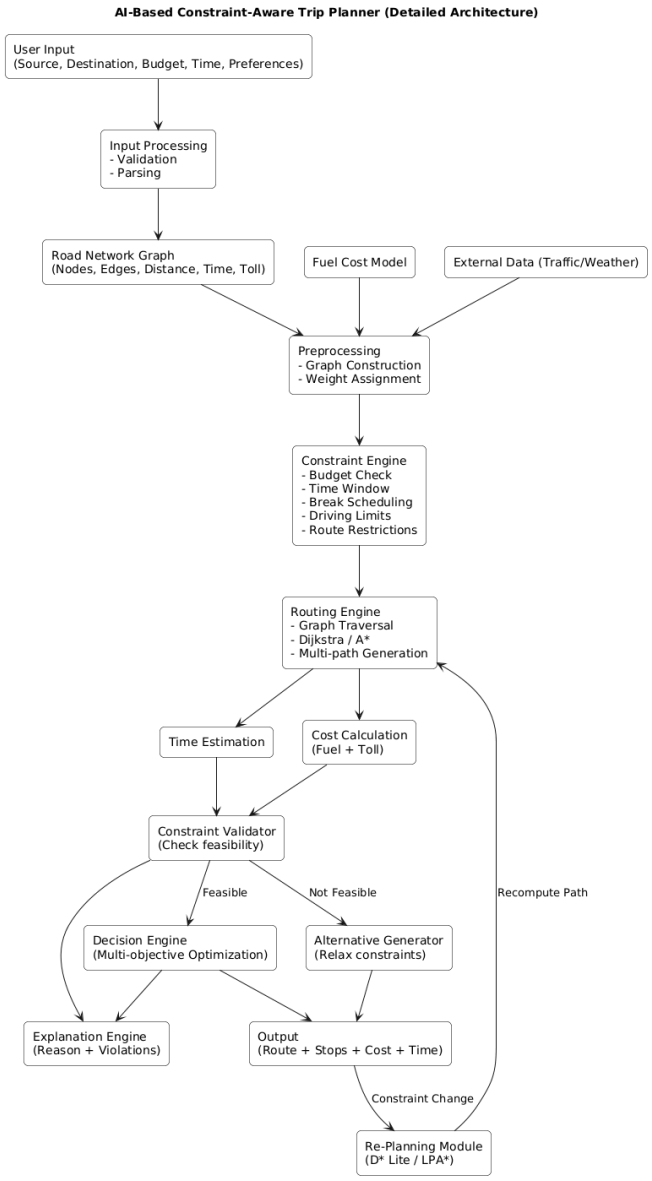

Fig. 1. Detailed Architecture of the Proposed System

D. Constraint Engine

Rules set by users, like spending limits or how long a trip can take, shape the route choices. Built right into the way paths are calculated, these rules draw from methods used in solving restricted shortest path challenges [6].

E. Routing Engine

Starting at the core, path calculations happen through Dijkstra's method [8]. As it moves along, each route gathers expense and duration together. Only options meeting set limits make it into the final list.

F. Evaluation Module

A single path gets checked by looking at how much it costs and how long it takes. Because of that, different workable

paths can be lined up side by side. Choices now weigh both money and minutes without leaning too hard on just one. That way, picking a route isn't about only speed or price alone. [3].

G. Constraint Validator

Every calculated route must follow the rules set by the user - no exceptions. Paths that break those rules get removed on the fly. Only valid options survive the process.

H. Alternative Route Generator

When a workable route isn't uncovered, trying fresh options begins - navigating through varied connections across the network without breaking set limits. Different turns get tested if the first way fails, keeping rules intact throughout the search process. Paths shift and bend under conditions, yet movement continues where possible inside the structure. Exploration carries on even when blocked at first, always checking nearby branches that obey boundaries.

I. Output Module

Travel plans show the best path, overall expense, expected duration, along with points in between. With these details, people can choose trips more wisely.

J. System Workflow

The overall workflow of the system can be summarized as follows:

1) Accept user inputs (source, destination, constraints)
2) Construct and preprocess the graph
3) Apply constraint-aware routing using Dijkstra's algorithm
4) Validate feasibility of generated routes
5) Select and return the optimal route

IV. Methodology

The proposed system follows a structured approach that integrates graph-based routing with constraint validation and multi-objective optimization to generate feasible and efficient inter-city travel routes.

A. Graph Modeling

The transportation network is represented as a weighted graph $G = (V, E)$, where nodes represent cities and edges represent road connections. Each edge is associated with attributes such as travel time, distance, and cost. This modeling approach is widely used in routing systems and is based on classical shortest path formulations [1], [4].

B. Shortest Path Computation

The system uses Dijkstra's algorithm to compute the shortest path between source and destination nodes [1]. Additionally, the A* algorithm is considered to improve efficiency by incorporating heuristic-based search, particularly for large-scale networks [5].

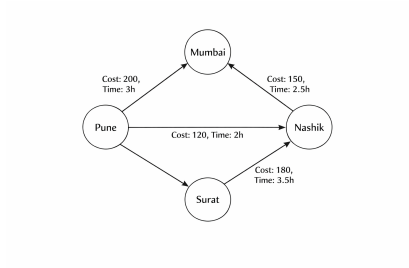

Fig. 2. Weighted graph representation of the transportation network where nodes represent cities and edges represent routes with associated cost and travel time.

C. Constraint Handling

Real-world constraints such as budget limits, travel deadlines, and driver fatigue are incorporated during path exploration. These constraints are enforced using principles from resource-constrained shortest path problems, ensuring that infeasible paths are eliminated early in the computation process [2].

D. Cost and Time Evaluation

Each candidate route is evaluated based on total travel time and total cost, which includes fuel expenses and toll charges. This evaluation enables the system to compare multiple feasible paths and determine the most efficient route.

E. Multi-Objective Optimization

Solving the route involves weighing costs alongside time, not just one at a time. Balance matters - some answers manage both well, even when they clash. These balanced choices show up through Pareto methods, revealing outcomes neither goal can improve without hurting the other. Each step forward nudges one value while pulling back on another. Trade-offs become clear only once competing needs face equal weight. Choices emerge not by picking sides but letting tensions shape results. Improvement hides in compromise, not dominance. Outcomes stay fair because pressure stays shared. The path forward bends where priorities meet resistance. Solutions appear when neither factor wins completely. [3].

F. Feasibility Validation

When the system calculates a path, it runs through a check to see if every rule set by the user holds true. Should any rule be broken, new options emerge - not by ignoring rules completely but bending some just enough to make things work. Paths adjust subtly when needed, yet still stay close to what was originally asked.

G. Output Generation

The best path shows up along with how long it takes, what it costs, also where you stop halfway. With these details people pick trips smarter.

V. Algorithm Design

This method builds on Dijkstra's original idea but adds limits like cost and duration when picking routes [8]. Instead of ignoring restrictions, it checks them each time a new point opens up. What sets it apart? It only moves forward if conditions still hold. Paths get longer, yet stay within set rules. Each step respects what travelers can actually afford or allow. Not every connection gets used - just those that fit. Finding the best way now means juggling more than distance alone.

Instead of chasing just one measure like classic methods do, this technique brings in real-world conditions right when checking new points. Because of that, paths failing to meet preset thresholds get ruled out early. Routes staying within bounds keep moving forward through checks. That way, what comes out fits better with how people actually plan journeys [6].

A. Problem Definition

Given a graph $G(V, E)$ where:

- V represents locations (cities or points of interest)
- E represents routes between locations

Each edge (u, v) is associated with:

- Cost $c(u, v)$
- Travel time $t(u, v)$

Input: Source node s, destination node d, budget B, maximum time T_{max}

Output: Optimal feasible path from s to d

B. Algorithm Steps

1) Initialize $dist[v] = \infty$ and $time[v] = \infty$ for all $v \in V$
2) Set $dist[s] = 0$ and $time[s] = 0$
3) Initialize priority queue Q and insert $(s, 0)$
4) While Q is not empty:
   a) Extract node u with minimum cost
   b) If $u = d$, terminate and return the path
   c) For each neighbor v of u:
      i) Compute:
         - $newCost = dist[u] + c(u, v)$
         - $newTime = time[u] + t(u, v)$
      ii) If $newCost \leq B$ and $newTime \leq T_{max}$:
         - If $newCost < dist[v]$:
           - Update $dist[v] = newCost$
           - Update $time[v] = newTime$
           - Store predecessor of v
           - Insert $(v, newCost)$ into Q
5) If no feasible path is found, return failure

C. Key Features

- **Constraint-aware routing:** Boundaries stay intact as paths unfold [6]
- **Multi-criteria handling:** Cost plus time get weighed at once, fitting ideas from balancing many goals together [3]
- **Efficient pruning:** Eliminates infeasible paths early, reducing unnecessary computations

D. Complexity Analysis

Time Complexity:

$$O((V + E) \log V)$$

Space Complexity:

$$O(V)$$

E. Real-World Applicability

Most times, this way fits best for arranging quick journeys where cash and minutes are short. You will spot these strategies popping up across transit systems built on strict timetables. When real movement of vehicles gets factored in, picking routes tends to get sharper . [2] [7].

VI. Results and Analysis

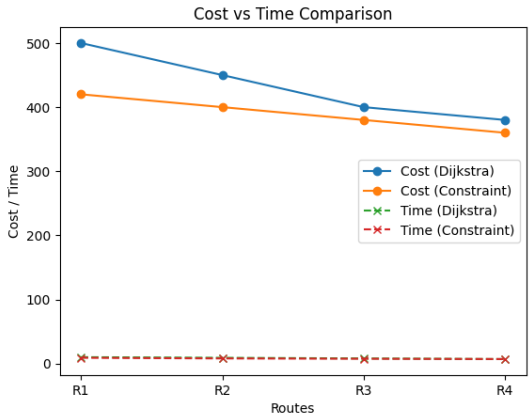

Fig. 3. Comparison of traditional shortest path and constraint-aware routing based on travel cost and time.

Analyzing the way routes are planned look for a method that takes into consideration both budget and time. Not just looking for the nearest point on a computer, this technique includes cost as time is going away [6]. Why is it different? Because means of transport are for doing two things that are very difficult simultaneously. Previous designs only focus on one goal, but this one wants to remain balanced. [8].

- **Improved constraint satisfaction:** Because of how it works, paths stick to budget and time limits without fail. This approach makes sure rules are followed every time. Handler showed this back in 1979. [6].
- **Efficient path computation:** Speedy route calculations emerge when Dijkstra's method gets extended - handling added constraints while staying fast [8].
- **Multi-criteria decision support:** Cost plus time get managed together, leading to stronger choices when picking options . [3].

It's noticeable that this approach performs strongly in actual conditions, particularly where individual needs are key - a definite improvement over earlier navigation methods [4].

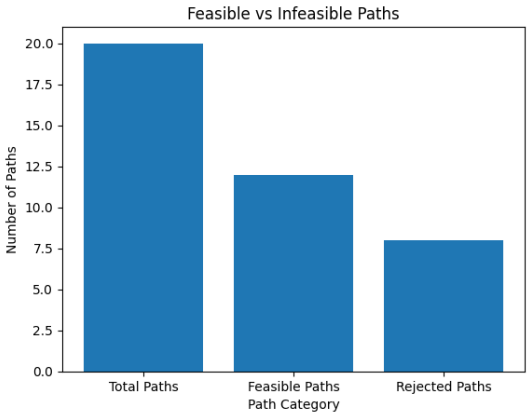

Fig. 4. Number of feasible and infeasible paths evaluated under budget and time constraints.

VII. Challenges Encountered

Halfway into crafting the route planner with boundaries in mind, surprises started appearing. With every hiccup, pausing to think became necessary before another step. Certain sections dragged on since answers didn't jump out right away. When limitations piled up, picking one design over another grew heavier. What felt minor early on shifted weight later. One at a time, movement stayed smooth. Past guesses cracked under actual pressure. Once flaws showed up through trials, changes came without force. Speed dropped, still understanding grew clearer later.

Most older path planners struggled when faced with multiple constraints such as price or time. Rather than focusing on a single target, handling more than one forced the system to use clever memory methods each time it evaluated another segment [6].

Speed dropped once rule checks entered the algorithm. Each fresh node faced several clearances before progressing - that caused drag. Staying near Dijkstra's original pace required adjusting task order and updates instantly. Success came not from adding checks but handling them invisibly. [8].

Most times when links between beginning and finish collapse, problems appear. Surviving those spots required crafting reactions that stand firm rather than fall apart. Staying upright through unrealistic demands shaped much of what exists now.

Tracking costs and timing became tough because adjustments needed to match perfectly through different sets of data. When edits missed their mark, errors crept into later calculations without warning. Paths built afterward would then fold under wrong assumptions.

Fixing issues in testing demanded effort, especially making sure each route obeyed the guidelines, no exceptions. That involved repeated cycles of corrections, followed by verifying stability under different conditions.

Yet things got messy fast once those issues came into play. How mixed up it turned out reveals what happens when classic graph tricks run headfirst into real-life walls.

VIII. Conclusion

Here comes a new way to plan trips when money and time matter. Built on top of Dijkstra's classic method, it adjusts step by step as limits appear. Instead of ignoring restrictions, they shape each decision inside the math itself. Feasible paths emerge simply because impossible ones never form. What results stays within set boundaries without extra checks later. Thinking ahead saves effort down the road. Route options respect reality right from the start. Limits guide movement like invisible rails. Only realistic journeys make it through. Hidden rules filter choices silently. Pathfinding adapts before mistakes happen. Boundaries define what counts. Choices shrink naturally under pressure. No detours beyond allowed zones occur. Logic bends around conditions given. Each turn obeys preset lines. Planning shifts with every restriction met. Outcomes follow built-in guardrails. Decisions grow out of enforced limits. Routes stay grounded at all times.

Most old-school route finders chase just one goal at a time. This new method juggles several goals together, fitting better into real-world travel choices. As it explores each next step, it checks rules on the fly. That way, dead-end options get tossed out early. Even with those extra checks, it runs about as fast as the standard ones do. Speed stays close to classic styles, despite added smarts along the way.

One way to see it: the setup builds solid paths within set boundaries, showing clear value. Because of this behavior, trip design and transit networks benefit when rules shape choices [4]. From start to finish, this approach offers a straightforward way to tackle route challenges that include limits and conditions. It connects classic path-finding ideas with how people actually need to move things in practice.

IX. Future Work

Even if the setup follows budget and schedule rules, small changes might make it run smoother or feel better when people use it.

Clicking things should feel natural right from the start. Choices pop up once a person begins picking how they like to travel - could be slow days, big adventures, or art-filled walks. As those options grow, so do preferences about rides and what each traveler can handle. The moment anything shifts, the map updates without delay. Twist after twist spreads through city maps when one small change appears. Paths grow right before your eyes, linking things together somehow. Life comes from instant replies on display - no lag, just movement. What you see shifts each time fingers move. Static shapes start flowing like water once touched. A single adjustment ripples down beneath. Because responses come right away, people keep interacting. The screen changes constantly, which holds focus tight.

Imagine feeding real-time traffic data into a system while storms roll in or prices shift - the path recalculates on its own. When downpours start, the journey bends without waiting. Roadblocks trigger immediate detours, smooth and automatic. Decisions gain clarity by reacting as events happen. Results

stop feeling random, instead moving with what's actually occurring.

Later on, looking back at old decisions builds smarter path advice for individuals. Because user habits get analyzed, recommendation systems improve gradually through experience [7].

What really makes a difference is how the app runs on cloud platforms, tied into huge streams of transportation details. This connection means it handles complex, real-world traffic networks smoothly, scaling up without lagging behind [4].

Maybe down the line, updated versions will balance several aims simultaneously - offering smoother travel, stronger safety, yet lighter environmental impact. This shift may help weave vehicles more smoothly into advanced trip planning systems [3].

Starting fresh, this overhaul aims at nothing short of change - transforming current systems into something more intuitive, fluid, built around travelers' needs. Rather than small fixes, deeper shifts guide how trips come together these days.

REFERENCES

[1] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," 1959.

[2] M. Desrochers and F. Soumis, "Shortest Path Problem with Time Windows," 1988.

[3] M. Ehrgott and X. Gandibleux, "Multiobjective Optimization," 2004.

[4] H. Bast et al., "Route Planning in Transportation Networks," ACM Computing Surveys, 2016.

[5] P. Hart, N. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," 1968.

[6] G. Handler and I. Zang, "A Dual Algorithm for the Constrained Shortest Path Problem," 1979.

[7] Y. Zheng, "Trajectory Data Mining: An Overview," ACM Transactions on Intelligent Systems, 2016.

[8] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," 1959.